THEME: VIS + AI · INFORMATION VISUALIZATION TERM PROJECT PROPOSAL

LagLens: What Does a Robot Actually Do While Its Policy Is Still Thinking?

A visual analytics tool for **action-chunk execution under inference latency**

The problem in plain words

1. A robot **policy** (a neural network) looks at the camera and plans the next 16 moves at once: an **action chunk**.
2. The robot carries out the first *h* moves without looking again (**open-loop**; *h* = **execution horizon**).
3. Planning the next chunk takes time *d* (**inference latency**, here 0–600 ms). Meanwhile the robot either
   • **waits** (**sync**): it stands still, or
   • **keeps moving on the old plan** and switches when the new one arrives (**async**). No waiting, but its moves come from an older camera picture (**stale actions**), and if the new plan does not continue the old one the robot **jumps** at the switch (**chunk-boundary discontinuity**). **Naive async** simply replaces the old plan; **inpaint async** makes the new plan continue it.

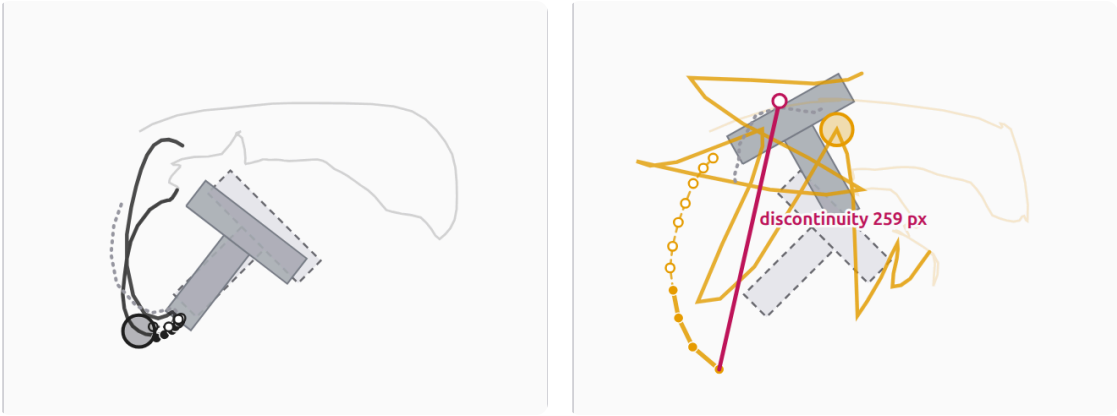
Why a visualization? Today the strategy, *d* and *h* are chosen from a success-rate table, e.g. “naive async, d = 4: 62 % vs. sync, d = 0: 72 %”. The table cannot say **which** situations broke, **when**, and **whether waiting, stale moves or jumps** caused it.

Our idea: same start, different schedule. We run one pretrained policy from the **same 50 start states** under **46 schedules** (strategy × *d* × *h*) and record every planned chunk, so any two schedules can be compared start by start and replayed move by move.

Background: Diffusion Policy, ACT, π0, SmolVLA (chunked policies) · stale actions: VLASH 2025 · discontinuities: BID 2025, SEAM 2026 · async inference & inpainting: RTC 2025, LeRobot docs. We found no visual analytics work on this problem (IEEE VIS / EuroVis / CHI search).

A real case from our data: same start (#1035), same delay (400 ms), h = 4

The robot pushes a T-shaped block onto a T-shaped goal (seen from above). Only the switching rule differs.

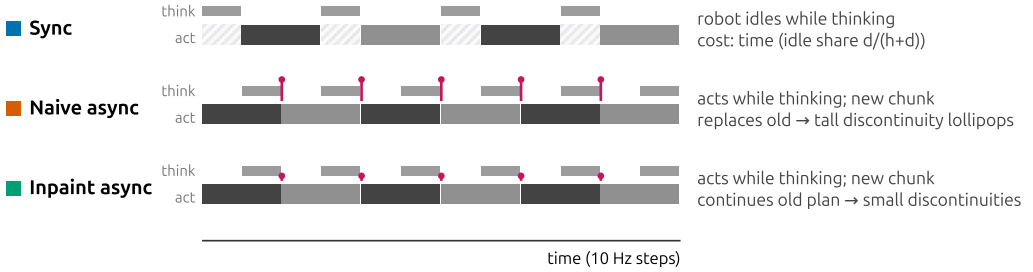


Inpaint async ✓ success
new plan continues the old one

Naive async ✗ failure
a 259 px jump at a plan switch knocks the block away

grey T = block · dashed T = goal · circle = pusher · line = path · both runs are equally stale (5.5 steps on average): the difference is the jump.

The three strategies over time (d = 2, h = 4)



think = policy computing a chunk · act = robot moving (tint changes per chunk) · hatched = robot idle · lollipop = jump at a switch

Self-generated rollout logs of a pretrained diffusion policy

Task (PushT, simulated, seen from above): a round pusher, the *agent*, must push a T-shaped block onto a fixed T-shaped goal. **Policy**: pretrained Diffusion Policy (`lerobot/diffusion_pusht`), a neural network that turns the last 2 camera frames into a chunk of 16 actions (2-D target positions of the agent) by iteratively removing noise from a random start (10 denoising steps, DDIM, CPU). Because each chunk starts from random noise, re-running an identical schedule can change an outcome (→ *noise replicate*). Same 50 initial states × (46 schedules + 1 noise replicate) = **2,350 episodes** (rollouts = recorded runs); every predicted chunk and every executed action is logged.

Attribute (episode table, 2,350 items × 19 attributes)	Type	Order / range / levels	Meaning
strategy KEY	nominal	sync · naive · inpaint	how a new chunk replaces the old one
latency d KEY	ordinal (ratio, steps)	0 < 1 < 2 < 4 < 6 steps (0–600 ms)	inference delay; d = 0 only for sync
horizon h KEY	ordinal (ratio, steps)	2 < 4 < 6 < 8	actions executed open-loop per chunk
replicate KEY	nominal (binary)	0 · 1	1 = sync d0 h8 re-run with new sampling noise (noise floor)
seed KEY	nominal (ID)	1000–1049 (50 levels)	initial state of block & agent (paired)
success	nominal (binary)	0 = fail, 1 = success	coverage > 95 % within 300 steps
coverage_final	quant. ratio, ↑ better	0.00–0.99	overlap of T-block with goal
steps_to_success	quant. ratio, ↓ better	68–300 (missing if failed)	time to finish
seam_gap_mean / max	quant. ratio (px), ↓	2.3–222.2 / 6–305	chunk-boundary discontinuity: jump between old and new plan at a hand-over
staleness_mean	quant. ratio (steps), ↓	0.5–9.5	age of the observation behind each executed action
idle_frac	quant. ratio, ↓	0.00–0.75	share of steps with nothing to execute
jerk_mean · path_len · n_inferences	quant. ratio	1.2–221.3 · 205–12828 px · 8–150	smoothness (jerk = 2nd difference of actions) · distance travelled · # chunks computed

Dataset type: multidimensional **table** of episodes, each linked to two **temporal** tables and to a **spatial** field (512 × 512 px workspace).

- **Step table** (598,117 items) · keys: schedule, seed, t (ordinal, 10 Hz) · values: agent x, y; block x, y (interval, px); block θ (cyclic, rad); action x, y (interval, px); chunk id (ordinal); index j 0–15 (ordinal); age (ratio, steps since its observation); held (binary, 1 = idle step); coverage (ratio 0–1, block–goal overlap).
- **Chunk table** (119,373 items) · keys: schedule, seed, chunk id · values: t_obs (when the observation was taken), t_avail (when the chunk was ready), executed window, 16 predicted (x, y) actions.
- **Size**: 215 KB episode CSV + 112 MB step/chunk JSON (lazy-loaded per schedule).

Keys vs. values: keys = (strategy, latency, horizon, replicate) → *schedule*; × seed → episode; × t → step; × chunk id → chunk. Everything else is a value.

Novel data: no public dataset logs per-inference chunks (survey, Oct 2026). Generated by us; scripts in `scripts/`. 1 step = 100 ms.

Model: Chi et al., Diffusion Policy (2023), checkpoint by Hugging Face LeRobot · PushT env: gym-pusht

Two people who must decide how a policy is executed

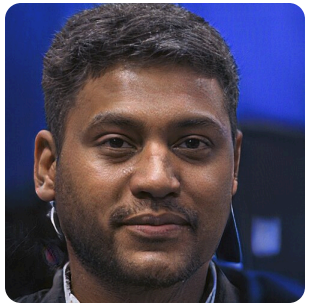


PRIMARY PERSONA

Mara Jensen

28 · 4th-year PhD student, robot learning lab · Copenhagen, Denmark

- **Context**: trains diffusion / flow-matching policies (generative models that output action chunks); deploys them on a real arm while the GPU server sits across the building (60–250 ms Wi-Fi latency).
- **Goals**: pick an execution strategy and horizon that keep success high at the latency she actually has; explain *why* in a paper.
- **Needs**: per-episode evidence of what failed, aligned with what the policy planned at that moment.
- **Pain points**: only success-rate tables and video; cannot tell stale actions from discontinuities; re-running real robot trials is costly; matplotlib debug plots show one episode at a time.



SECONDARY PERSONA

Arjun Mehta

37 · ML platform engineer, warehouse-robotics start-up · Seoul (Pangyo), Korea

- **Context**: owns the policy-serving software (the robot sends camera frames to a GPU server, which returns action chunks); does not train models.
- **Goals**: set a latency budget (the largest inference delay a robot may tolerate) per robot and a safe default strategy; know when to fall back to sync.
- **Needs**: an overview of how success degrades with latency for each strategy, with confidence that it is not a few lucky seeds.
- **Pain points**: research papers report curves on other robots; no shared vocabulary with researchers about “why it jitters”.

Difference	Mara (primary)	Arjun (secondary)
Expertise	Policy internals (diffusion, chunking)	Systems, networking, latency
Granularity of questions	Single episodes, single chunk hand-overs	Schedules and aggregate trends
Main view used	Workspace replay + execution timeline, seed shifts (T3, T4)	Latency-response curves + paired seed table (T1, T2)
Decision made	Which strategy / horizon to propose and why	Latency budget and fallback rule

Portraits: StyleGAN-generated, public domain (Wikimedia Commons “Woman 1.jpg”, “Man 2.jpg”); the people do not exist. Personas are fictional.

Four tasks, from schedules down to a single discontinuity

#	Action + target	Concrete question (Mara)	Why	Expected outcome
T1	**Compare → trends** of success / boundary discontinuity / staleness across schedules	"How fast does success fall with latency for each strategy, and which horizon is most robust?"	Choose a candidate schedule for the latency the lab actually has.	A shortlist of robust schedules and a latency budget per strategy.
T2	**Locate → outliers**: initial states whose outcome **flips** between a reference and a candidate schedule	"Going from sync d=0 to naive async d=4, *which* seeds start failing, and which are rescued?"	Aggregate rates hide which situations break; paired seeds isolate the schedule's effect.	A small set of fragile seeds to inspect (and to add to a regression test set).
T3	**Identify → features** in time & space: where stale actions, discontinuities and idling occur in a flipped episode	"At which step does the candidate diverge from the reference, and was the executed action stale, a discontinuity, or a wait?"	Diagnose the mechanism of failure, not just its rate.	A causal hypothesis, e.g. "a 30 px discontinuity right at block contact makes the push slip."
T4	**Discover → correlation** between boundary discontinuities (plan inconsistency) and task outcome (coverage) over paired episodes	"When a seed moves from R to C, do larger discontinuities go with lower coverage? Does inpainting shrink the discontinuities?"	Recent work claims failed episodes show larger chunk-boundary artifacts (Wang 2026); check it on our policy.	Evidence for or against a strategy (direction and size of shift per seed).

Arjun mainly needs **T1** (latency budget) and **T2** (is the drop driven by a few seeds?). **Mara** chains T1 → T2 → T3 → T4 in one session: overview first, then drill down to a single chunk hand-over (details on demand).

Each view answers one task with the most effective idiom

View	Task	Idiom & why (VIS literature)	Interaction
A · Latency Response	T1	*"How fast does each strategy get worse as the policy gets slower?"* **Line chart small multiples**: one panel per horizon h, x = latency d on its true scale, y = metric, strategies **superposed** (hue + shape). Position on a common scale beats the luminance of a heatmap [Cleveland & McGill 84; Heer & Bostock 10]; superposition for direct sync-vs-async comparison, juxtaposition across h [Gleicher 11]. **95 % interval** lines instead of bars [Correll & Gleicher 14]; hover = paired difference to R.	Click mark = C (compare), shift-click = R (baseline); **one question bar**: Success · Idle (cost of sync) · Staleness (cost of async) · Discontinuity (cost of naive async), drives A *and* B; recomputes on seeds selected in B.
B · Paired Seed Table	T2, T4	*"Which start states flip between R and C, and is that more than chance?"* **Composite Table-Lens view** [Rao & Card 94; Javed & Elmqvist 12]: one row per seed shared by (left) an outcome matrix — filled = success, R column black, C column orange — and (right) **dumbbells** R → C of the attribute that belongs to A's question. Rows are **grouped by R→C change** (spatial region), so flips break their band's pattern; a **re-run** column shows chance flips (11/50).	No controls of its own. Click column header = C; click cell = open episode; **brush** dumbbells or click a band → **cross-filter** A [Weaver 10].
C · Workspace Replay	T3	*"Watch both runs side by side: what was planned, what was done, where it jumped."* Juxtaposed R | C maps at identical scale (spatial data on true geometry). Only three overlays: executed part of the current chunk (solid), its thrown-away part (dashed), the chunk still being computed (dotted) and the latest **discontinuity** (magenta). Status line in strategy terms ("idle: waiting for inference").	Play / scrub / ← → (**navigate** t); hover a planned point → when planned / executed; click → jump.
D · Execution Timeline	T3	*"When was the policy thinking, which plan did each move come from, where were the jumps?"* **Gantt lanes** on a shared linear time axis [Plaisant 96; Brehmer 17]: per run a **think** track (inference) and an **act** track (chunk spans, hatched = idle), discontinuities as **lollipops** (length), coverage lines below. Sync shows think/act *alternating*, async *overlapping*.	**Brush** a time window → highlighted path in C; hover chunk → its plan in C; click → cursor.

Why this is not a generic dashboard: C and D trace every executed move back to the plan (chunk) and the camera frame it came from, which no chart tool offers; A and B compare two schedules *start state by start state* against a chance baseline (re-run), not just averages. **Less is more**: the first draft had 5 views and ~10 hues; following the rules of parsimony and consistency [Wang Baldonado 00] we merged the two seed-level views, gave every hue one meaning, and cut ten technical controls to **one question bar** named after who pays each cost.

The most important attribute gets the most effective channel

Attribute (type)	View	Mark	Channel	Justification
success rate, idle, staleness, boundary discontinuity (ratio)	A	point + line	position y (common scale)	best magnitude channel; slope = degradation
latency d (ratio)	A	—	position x (true spacing)	100 ms steps, not ordinal bins
horizon h (ordinal)	A	panel	spatial region (small multiples)	juxtaposed, shared y
strategy (nominal, 3)	A, B	line, point	hue + shape (●■◆)	redundant: never hue alone
uncertainty (95 % interval)	A	thin line	position (extent)	avoids within-the-bar bias
success per seed (binary)	B	cell	luminance (filled / empty)	2 levels → 1 luminance step
R→C change (nominal, 4)	B	row band	spatial region	top categorical channel
attribute under R and C (ratio)	B	dumbbell	position x (symlog if long-tailed)	aligned shift, no crossing
agent / block / plan (interval, px)	C	line, point, polygon	position (x, y)	spatial data, true geometry
executed / discarded / computing	C	line, point	line style + fill	separable, keeps hue for identity
think, act intervals (time)	D	bar	position + length on time axis	alternation vs overlap visible
boundary discontinuity (ratio, px)	C, D	lollipop, segment	length + magenta hue	magnitude → length
coverage (ratio)	D	line	position y	R vs C on a common scale

Hue budget — one meaning per hue

■ Reference R ■ Candidate C

● Sync ■ Naive async ◆ Inpaint

■ Discontinuity (C, D) grey: everything else

Okabe–Ito hues. CVD-checked with Machado et al. (VIS 2009) in OKLab: R vs C ΔE ≥ 48; strategy hues ≥ 9 plus redundant shape.

Ranking used (Munzner, Ch. 5):
magnitude: position on common scale > length > angle/area > luminance
identity: spatial region > hue > shape

Scalability: A 47 points · B 2,350 cells + 50 dumbbells · C/D one episode (≤ 300 steps, ≤ 150 chunks) — SVG at interactive rates.

Cleveland & McGill (1984) · Heer & Bostock (CHI 2010) · Gleicher et al. (2011) · Correll & Gleicher (VIS 2014) · full list: docs/design_rationale.md

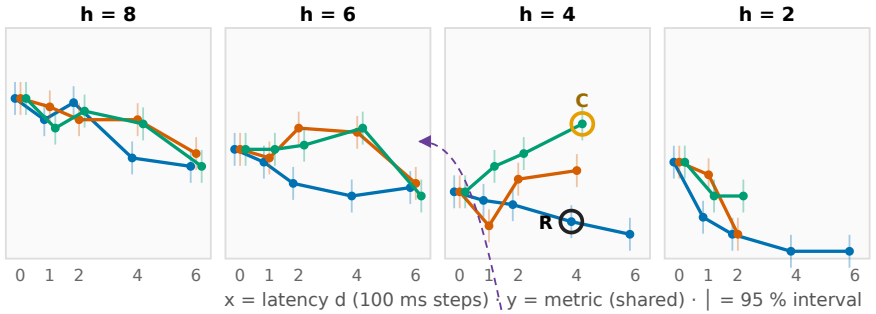
LagLens

pick / swap R and C anywhere

● R baseline Sync · d=4 · h=4 ▾ ⇄ ● C compare Inpaint · d=4 · h=4 ▾ ? ↺

A · Latency Response (T1)

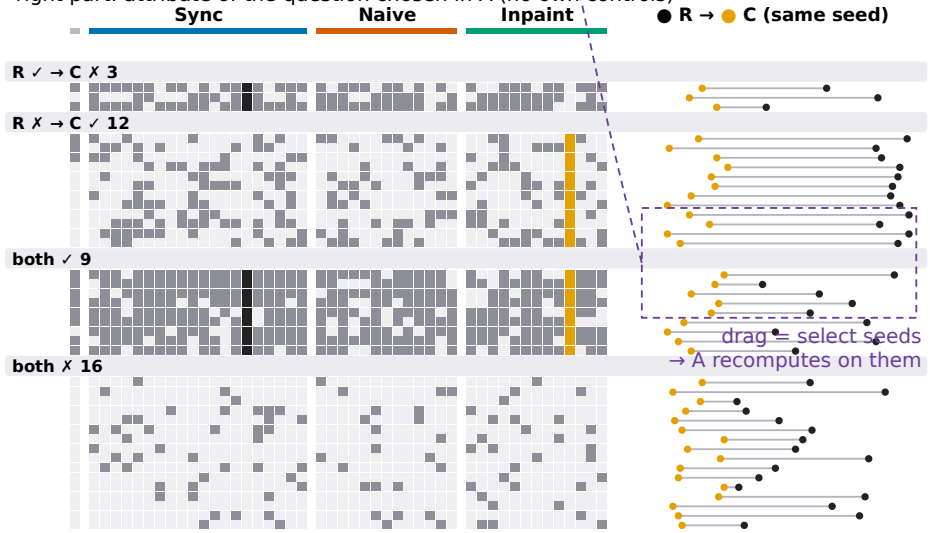
[**Success** | Idle | Staleness | Discontinuity] ● Sync ■ Naive ◆ Inpaint



question bar sets A and B · click mark = C · shift-click = R · hover = value & Δ vs R

B · Paired Seed Table (T2, T4)

right part: attribute of the question chosen in A (no own controls)



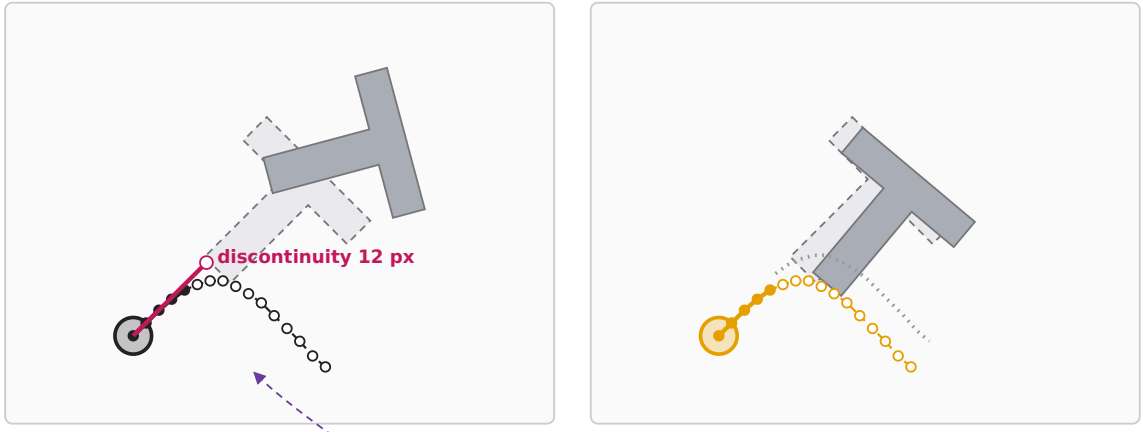
click cell = open episode (column → C) · click header = C · click band = select its seeds

C · Workspace Replay (T3)

hover planned point = when planned / executed · click = jump · ← → space

R · Sync d4 h4 ✗ · idle: waiting for inference

C · Inpaint d4 h4 ✓ · acting, 4 steps stale · thinking

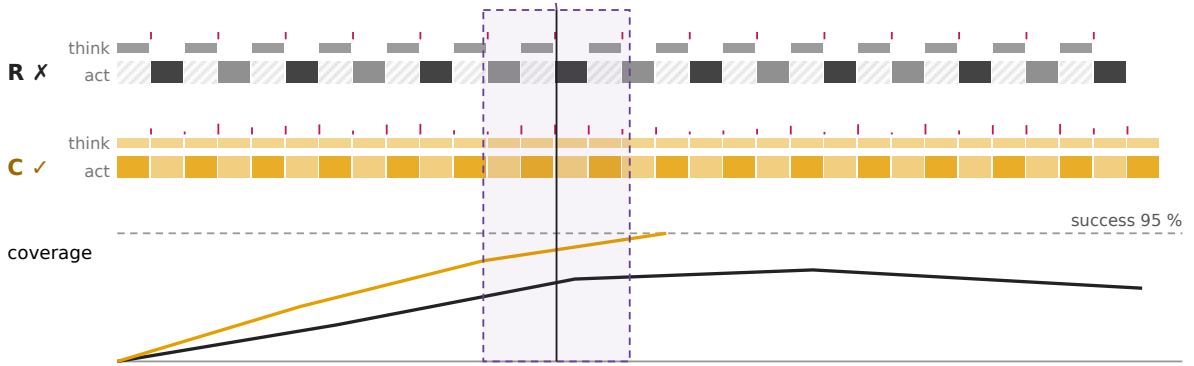


— executed part of chunk ┄ thrown away ⋯ next chunk computing ━ boundary discontinuity

▶ play

D · Execution Timeline (T3)

think = inference · **act** = chunk executed (hatched = idle) · lollipop = boundary discontinuity



drag = time window (→ path highlighted in C) · hover chunk = its plan in C · click = cursor

Flow: A (pick R and C on the latency curves) → B (find flipped seeds, check the R→C shift, select seeds → A recomputes) → C + D (where and when it went wrong). d3.js v7 only.